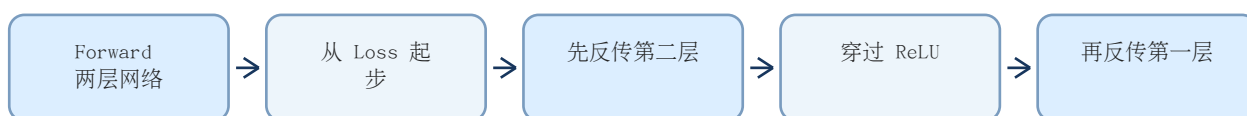


## 第25讲核心概念速查卡

### 两层 MLP 的矩阵 Backpropagation

核心概念：速查卡

本讲把上一讲的单神经元 Backpropagation 升级为真正的两层 MLP。目标是看懂每一层的 Tensor、Gradient、Transpose 和 Shape 怎样配合，让最终 Loss 一层一层传回  $W2$ 、 $b2$ 、 $W1$ 、 $b1$ ，最终还能继续传给输入。



建议学习方式：先跟着固定的 3->2->1 数字例子手算，再理解矩阵公式，最后运行 Numerical Gradient Check 与 PyTorch Autograd 对照。

## 1. Forward

```
z1 = x @ W1 + b1
a1 = ReLU(z1)
y_hat = a1 @ W2 + b2
L = (y_hat-target)^2
```

## 2. Backward

```
dy_hat = 2(y_hat-target)
dW2 = a1^T @ dy_hat
db2 = sum(dy_hat)
da1 = dy_hat @ W2^T
dz1 = da1 * ReLU_prime(z1)
dW1 = x^T @ dz1
db1 = sum(dz1)
dx = dz1 @ W1^T
```

### 3. 本讲数字答案

```
x=[1,2,-1], z1=[3,-2], a1=[3,0]
y_hat=6.5, target=5, Loss=2.25
dy_hat=3
dW2=[[9],[0]], db2=[3]
da1=[6,-3], dz1=[6,0]
dW1=[[6,0],[12,0],[-6,0]]
db1=[6,0], dx=[3,6,-3]
```

## 4. Linear backward 通用模板

$$Y = X @ W + b$$

$$dW = X^T @ dY$$

$$db = \text{sum}(dY \text{ over batch})$$

$$dX = dY @ W^T$$

## 5. 必须记住

参数梯度 Shape 必须与参数 Shape 一致。

Backward 顺序与 Forward 相反。

ReLU backward 是逐元素 gate。

多样本时参数梯度会聚合各样本贡献。

mean / sum reduction 会改变梯度尺度。

Transpose 来自连接关系，不是为了凑 Shape。

## 6. PyTorch 映射

```
loss.backward()  
print(W2.grad)  
print(b2.grad)  
print(W1.grad)  
print(b1.grad)
```

核心概念：一句话

Backpropagation 就是从 Loss 出发，按计算图逆序，把上游梯度通过每层的局部 Jacobian/局部规则继续传回去，同时得到该层参数的梯度。